Online Video Meeting Application

1. Project Description:

MeetEx is a full-stack, real-time online video meeting application built using React.js on the frontend and Node.js with Express.js on the backend, supported by a MongoDB database for user management and JWT-based authentication for secure access. The platform leverages the Agora RTC SDK to enable seamless peer-to-peer and multi-party video and audio communication directly in the browser.

2. Scenario :

Consider a scenario where a remote team of professionals or a group of students needs to collaborate in real time without relying on heavyweight commercial tools. A user visits MeetEx, registers an account with their name and email, and logs in to receive a secure JWT token. From the Home dashboard, the user either creates a new meeting by entering a custom room name, or joins an existing session by entering a shared room code.

3. System Requirement :

3.1 Software Requirements

- **React for the web interface**
- **Node with Express for the server**
- **MongoDB for storing user details**
- **JWT for secure login**
- **Agora RTC for video and audio**

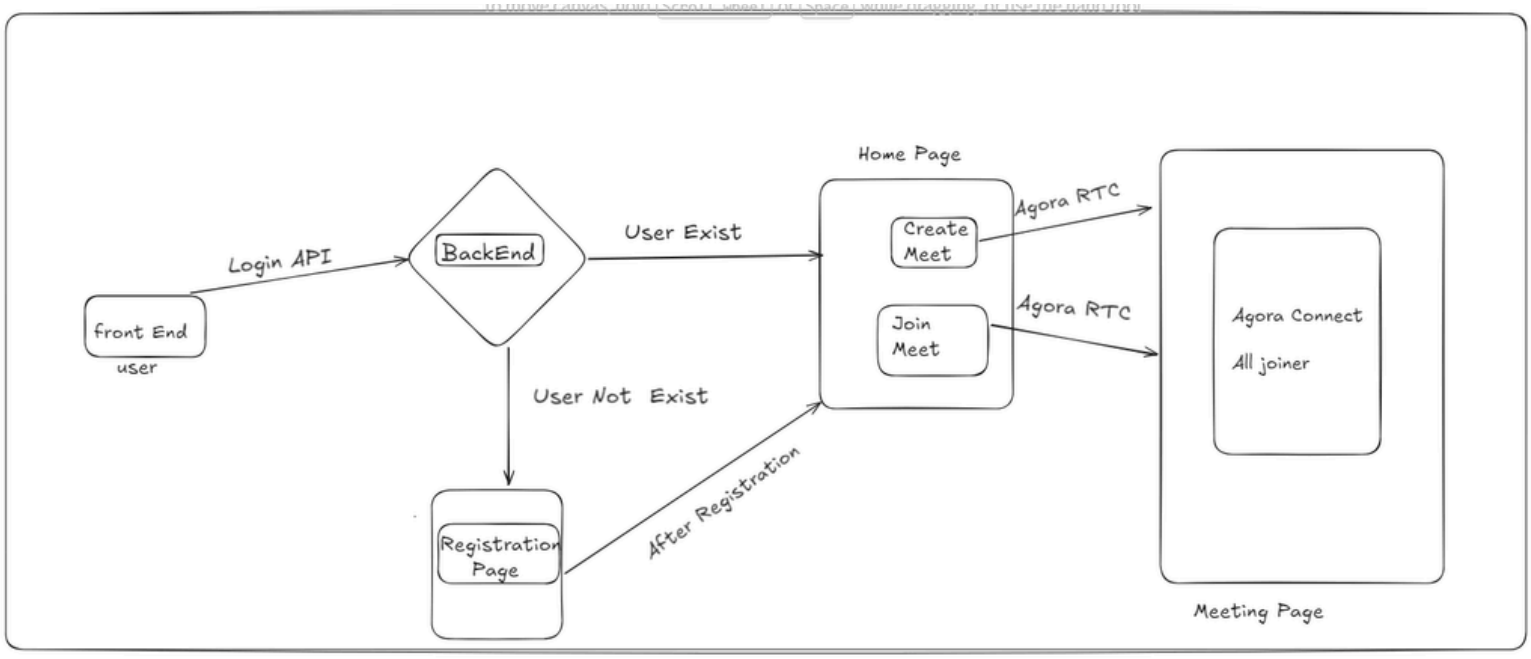
• **Tailwind for the look and layout**

3.2 Hardware Requirement :

- **RAM**
  Minimum **8 GB (16 GB recommended)** — to handle dev servers, IDE, and browser testing simultaneously.
- **Storage**
  Minimum **1 GB free space** — for **npm packages, MongoDB setup, and local project files.**
- **Display**
  1366×768 resolution or higher — for optimal coding experience and UI layout visualization.
- **Network**
  Stable broadband internet — required for **Agora RTC real-time audio/video streaming.**

4. Project Architechture :

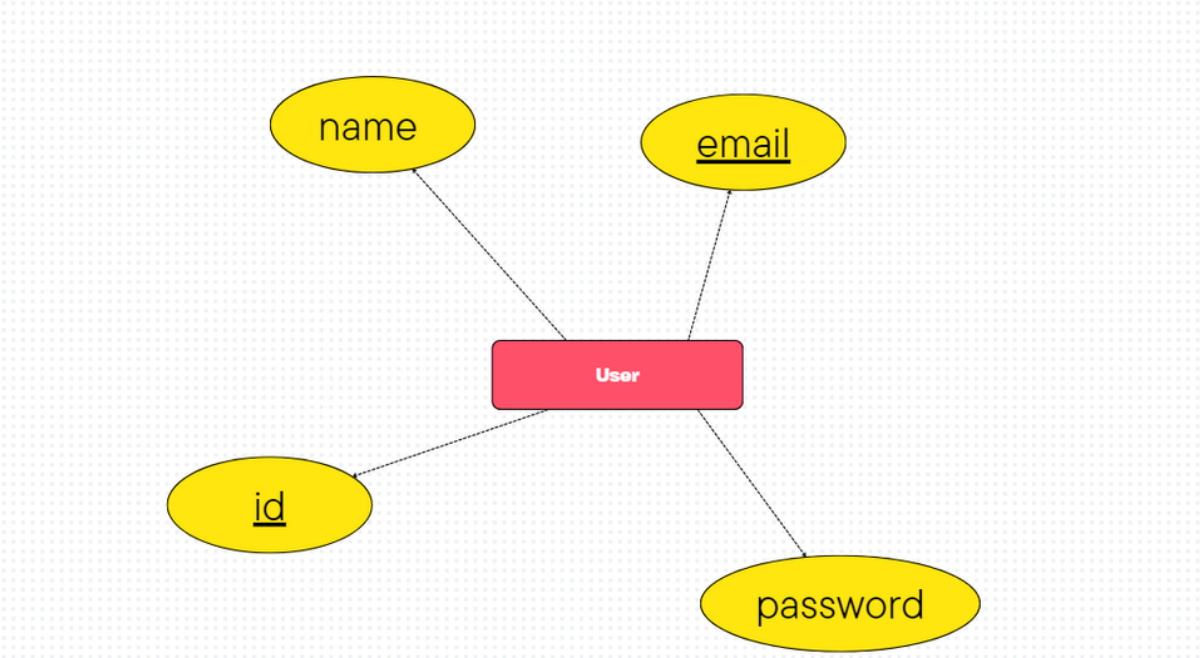
4.1 Technical Architecture :



- When a user visits the website, the system first checks whether the user is already registered or not. If the user is new, they are redirected to the **Registration Page** where they must create an account. After successful registration, the user is automatically redirected to the **Home Page** of the application.

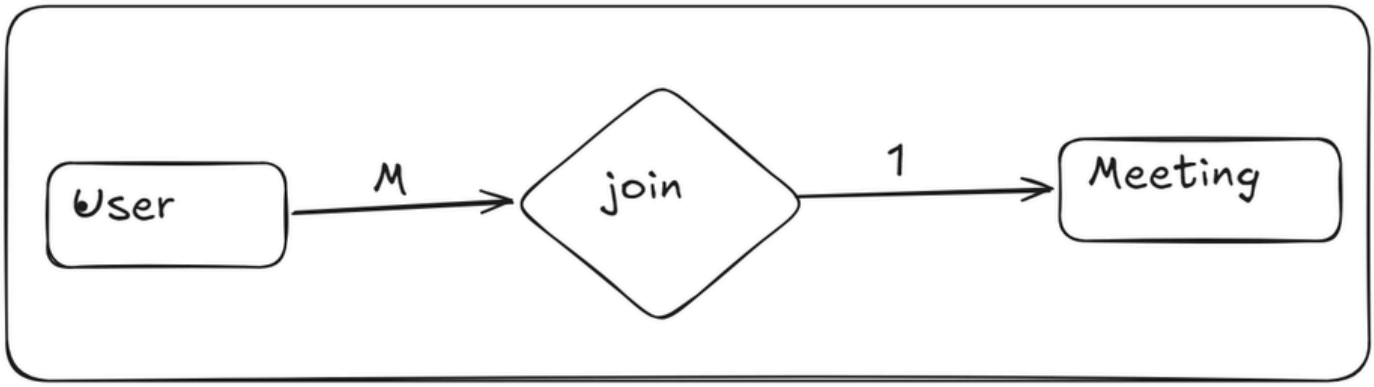
- On the Home Page, the user is provided with two options: **Create Meet** or **Join Meet**. To proceed, the user must enter a meeting name. Once the meeting name is entered, the user is redirected to the **Meeting Page**.
- On the Meeting Page, the user can access controls such as **video on/off** and **microphone on/off**, which allow them to manage their audio and video during the meeting session.
- If another user visits the website and follows the same process (registration if required, then entering the same meeting name on the Home Page), both users are connected automatically inside the same meeting room.
- The connection between users is established using **Agora RTC SDK**, which enables real-time audio and video communication. The Agora RTC service manages synchronization of participants and ensures smooth video conferencing between all users who join the same meeting room.
- This architecture ensures a structured workflow where the frontend handles user interaction, the backend verifies user authentication, and the Agora RTC integration provides real-time communication functionality between participants.

4.2 ER Model:



User Entity

Relation ship



Entity Relationship Model Description:

The ER model of the video conferencing system includes two entities: User and Meeting. Multiple users can join a single meeting room by entering the same meeting name. Therefore, a many-to-one (M:1) relationship exists between the User and Meeting entities. The meeting connection is established dynamically using the Agora RTC service for real-time communication.

4.3 Features:

- User Registration — New users sign up with name, email, and password (stored as bcrypt hash).
- User Login — Authenticates credentials and returns a signed JWT token for session management.
- Protected Routes — All pages beyond /login and /register require a valid JWT token in localStorage.
- Create a Meeting Room — Authenticated users enter a custom name to instantly create a new room.
- Join a Meeting Room — Users enter an existing room code to join an active live session.
- Live Video & Audio — Powered by Agora RTC SDK; multiple participants stream camera and mic in real time.
- Mute / Unmute Microphone — Toggle local audio track on/off during an active meeting.
- Stop / Start Camera — Toggle local video track on/off without leaving the room.

- Remote User Grid — Remote participants are auto-discovered and rendered in a responsive video grid.
- Clean Leave — Releases media tracks and properly disconnects from the Agora channel on exit.
- Responsive UI — Tailwind CSS utility classes ensure the layout adapts across screen sizes.

4.4 Roles and Responsibilities :

MeetEx currently supports a single user role — Authenticated User. All permissions are identical for every logged-in user.**MeetEx currently supports a single user role — Authenticated User. All permissions are identical for every logged-in user.**

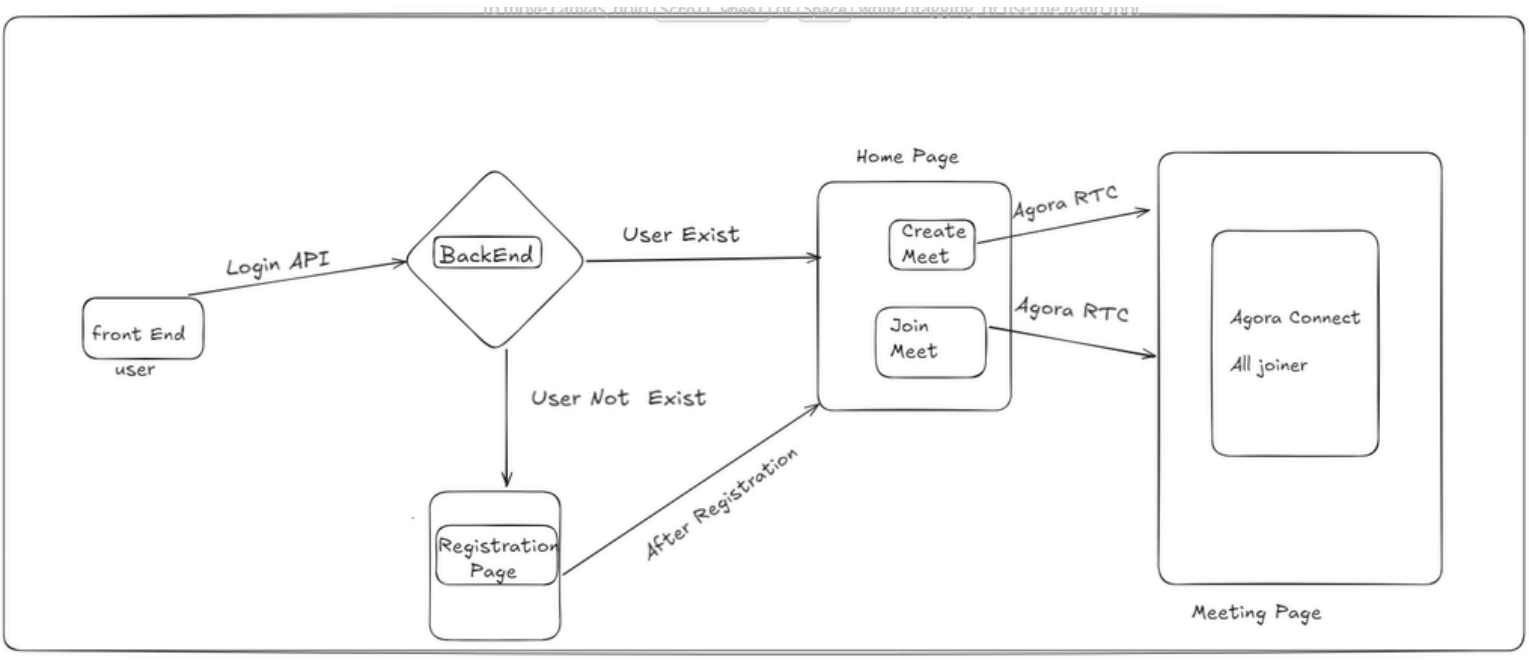
Role | Capabilities

Guest (unauthenticated)
Can access /login and /register pages only. Redirected away from protected routes.

Authenticated User

Can access Home dashboard, create/join meeting rooms, and use all in-call controls (mute, camera, leave).

4.5 User Flow



The following describes the complete user journey through MeetEx from first visit to leaving a meeting:

1. User visits the app → redirected to /login (ProtectedRoutes blocks unauthenticated access).
2. User clicks Register → fills name, email, password → POST /api/auth/register → account created.
3. User logs in → POST /api/auth/login → JWT token and name stored in localStorage → redirected to Home /.
4. Home page loads → user sees their name, two options: Create Meeting or Join Meeting.
5. Create Meeting: user types a room name → navigate to /meet/:roomName.

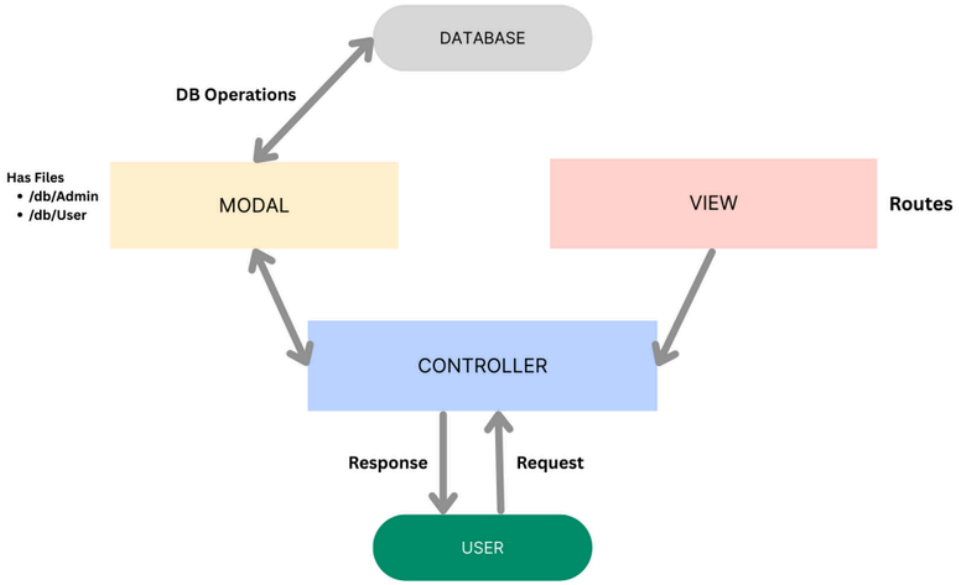
6. Join Meeting: user enters an existing room code → navigate to /meet/:roomCode.

7. MeetRoom page mounts → joinChannel() called → Agora client joins the channel, camera + mic tracks created and published.
8. Other participants who join the same room are auto-subscribed → their video appears in the remote user grid.
9. User can toggle Mute / Stop Cam at any time using the control bar.
10. User clicks Leave → tracks closed, Agora channel left, navigate back to Home.

11. User clicks Logout → token and name removed from localStorage → redirected to /login.

5. **MVC Pattern :**

MeetEx's backend follows the Model-View-Controller (MVC) architectural pattern — a design approach that separates application logic into three clearly defined, interconnected layers. This structure improves code readability, maintainability, and makes the application easy to scale or test independently at each layer.



Model Layer (Data Layer)

The Model layer handles all data-related logic: schema definition and database operations. In MeetEx, the User model is defined using Mongoose in /server/models/userModel.js. It specifies the shape of a user document — name (String, required), email (String, required), and password (String, required, stored as a bcrypt hash). All MongoDB queries (create, findOne, find) are executed through this model.

View Layer (Routing Layer)

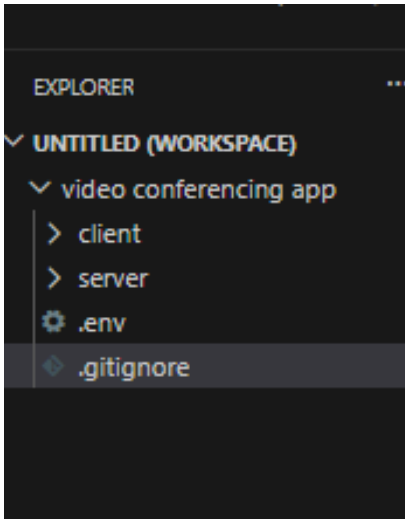
In a REST API context, the View is the routing layer. MeetEx defines its API endpoints in /server/routes/authRoute.js using Express Router. Two routes are registered: POST /api/auth/register and POST /api/auth/login. Each route maps directly to its corresponding controller function, keeping the routing layer thin and focused purely on URL-to-function mapping.

Controller Layer

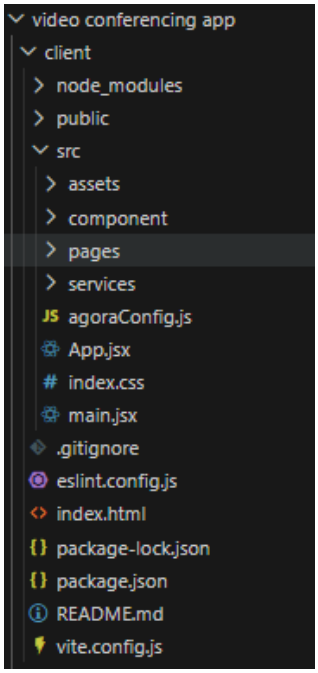
The Controller is the brain of the application. Located in /server/controllers/authController.js, it contains the register and login functions. On registration, it checks for existing email duplicates, hashes the password using bcrypt, and inserts a new User document. On login, it finds the user by email, compares the password hash with bcrypt.compare(), and — if valid — signs and returns a JWT token using jsonwebtoken. All business logic lives here, keeping models and routes clean.

6. Project Setup and Configurationp :

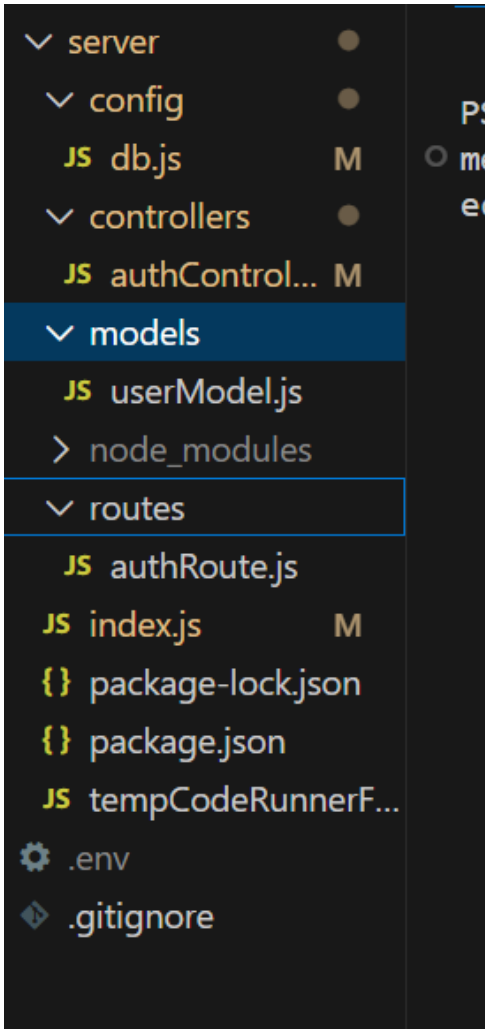
6.1 Folder Structure :



6.2 Client Folder Structure:



6.3 Server Folder Structure :



7. Project Execution :

Step 1 — Download and Open the Project

- Clone or download the project folder from the repository.
- Open the project root folder in VS Code.
- You will see two sub-folders: client (React frontend) and server (Node.js backend).

Step 2 — Set Up the Frontend (React)

a. Open a terminal and navigate into the client folder:

```
cd client
```

a. Install all required frontend packages:

```
npm install
```

a. Start the React development server:

```
npm run dev
```

The app should now be running at: http://localhost:5173

Step 3 — Set Up the Backend (Express)

a. Open a new terminal tab/window and navigate into the server folder:

```
cd ../server
```

a. Install all backend dependencies:

```
npm install
```

Step 4 — Configure Environment Variables

a. Inside the server folder, create a new file named `.env` (no file extension).
b. Add the following environment variables:

```
MONGO_URI=mongodb://localhost:27017/video_conferencing_app_db

PORT=5300

JWT_SECRET="tyagiG"
```

Step 5 — Start the Backend Server

a. Inside the server folder, start the backend using nodemon:

```
nodemon index.js
```

The server should start on: http://localhost:5300

Step 6 — Open the Application

Demo Video Link :
https://drive.google.com/file/d/1SyoWRCy2mxjliq3o1BwOnDZvROg7Q_-r/view

Code Link :https://github.com/chandantyagii/Conferencing-App